# Lecture 3
# Reading History and Undoing Mistakes

`log`, `show`, `diff`, `restore`, `commit --amend`, and `revert`

**Learning goals**

- Read the history of a project with `git log` and `git show`, and name commits with `HEAD` and `HEAD~n`.
- Compare versions with the three forms of `git diff`, and read a diff.
- Undo a mistake at the right stage: unstage, discard, amend, or revert.
- Tell *safe* undo commands from commands that destroy work.

## 1 Why this lecture matters

Version control earns its keep when something goes wrong. Everyone saves the wrong file, mistypes a message, or deletes a line they needed. Git gives you a tool for each situation, but the tools differ in one crucial way: some are completely safe, and one throws work away for good. Learning which is which turns Git from scary into reassuring. In this lecture, the main command that can lose something you did not commit is `git restore file`, and we will put a warning sign on it. (`git rm -f`, at the end, is the other one.)

We will use a small practice repository whose history is the same on every computer. From the course folder run:

```
$ bash codeHints/L03/setup_demo3.sh
$ cd practice/demo3
```

It creates `demo3` with four commits (a README and three recipes). Commit IDs are computed from the content, author and date, and the script fixes all of these, so the IDs printed in this lecture should be the ones you see. (On Windows with automatic line-ending conversion they may differ; everything else still works.) To get a fresh copy at any time, first leave the folder (`cd ../..`), then run `bash codeHints/L03/setup_demo3.sh --reset`, then `cd practice/demo3` again.

## 2 Reading the history

### 2.1 `git log`

```
$ git log
commit 659ed06370bf2b80289ead79663c5ce21ba67c61
Author: Practice User <practice@example.com>
Date:   Mon Mar 4 09:00:00 2024 +0000

    Add waffle recipe

commit c595297a5267368106009265a473686c8d8ec827
Author: Practice User <practice@example.com>
Date:   Sun Mar 3 09:00:00 2024 +0000

    Add crepe recipe
...
```

The newest commit comes first. Each entry shows the full 40-character ID, the author, the date and the message. If the output is longer than the screen, Git shows it in a *pager*: scroll with the arrow keys or space, and press `q` to quit.

Usually the compact form is enough:

```
$ git log --oneline
659ed06 Add waffle recipe
c595297 Add crepe recipe
906ca29 Add pancake recipe
9d99f7a Add README
```

A few useful options (they can be combined):

| Command | What it shows |
|---|---|
| `git log --oneline` | One line per commit: short ID and message. |
| `git log -n 2` | Only the 2 newest commits (also works as `-2`). |
| `git log --stat` | Also which files each commit changed, with counts of added and removed lines. |

## 2.2 `git show`: one commit in detail

`HEAD` means "the commit I am on now", here the newest one (Section 2.3 explains it).

```
$ git show HEAD
commit 659ed06370bf2b80289ead79663c5ce21ba67c61
Author: Practice User <practice@example.com>
Date:   Mon Mar 4 09:00:00 2024 +0000

    Add waffle recipe

diff --git a/recipes.txt b/recipes.txt
index 2d1a741..0104975 100644
--- a/recipes.txt
+++ b/recipes.txt
@@ -1,2 +1,3 @@
 Pancakes: flour, milk, eggs
 Crepes: flour, milk, eggs, butter
+Waffles: flour, milk, eggs, sugar
```

`git show` prints the message and then the *change* the commit made (a "diff", explained below). You can give it any commit ID: `git show c595297`. The short 7-character ID is enough as long as it is unambiguous.

## 2.3 Naming commits: `HEAD`, `HEAD~1`

You rarely want to copy IDs. Git has names for commits relative to where you are:

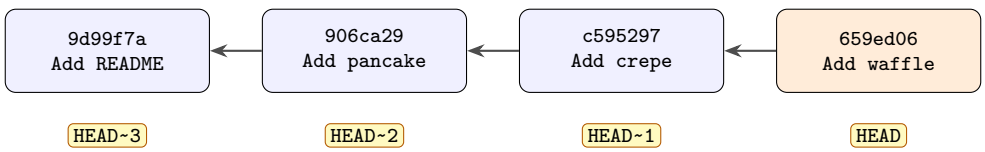



**Figure 1:** `HEAD` is the commit you are currently on (the newest one, for now). `HEAD~1` is its parent (the commit just before it), `HEAD~2` the one before that, and so on. Arrows point from a commit to its parent.

So `git show HEAD~1` shows the crepe commit. You will use `HEAD` constantly: it means "where I am now".

**Try it**

In `demo3`, run `git log --oneline`, then `git show HEAD~2`, then `git log --stat -n 2`. Which file did the very first commit create? (Use `git show 9d99f7a`.)

## 3 Comparing versions: `git diff`

A **diff** lists the differences between two versions. Remember the three areas from Lecture 2: `git diff` compares different pairs of them.

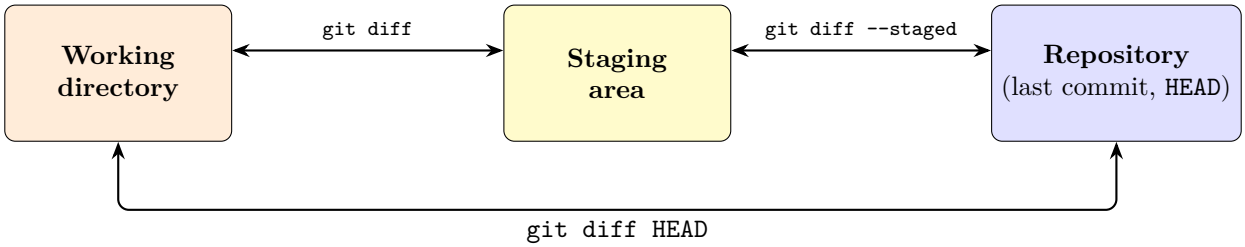



**Figure 2:** Which areas each form of `git diff` compares.

Make a change to see them in action:

```
$ echo "Muffins: flour, milk" >> recipes.txt
$ git diff
diff --git a/recipes.txt b/recipes.txt
index 0104975..11161ec 100644
--- a/recipes.txt
+++ b/recipes.txt
@@ -1,3 +1,4 @@
 Pancakes: flour, milk, eggs
 Crepes: flour, milk, eggs, butter
 Waffles: flour, milk, eggs, sugar
+Muffins: flour, milk
```

**Reading a diff.**

- `--- a/recipes.txt` is the old version, `+++ b/recipes.txt` the new one.
- A line starting with `+` was *added*; a line starting with `-` was *removed*; lines with a space in front are unchanged context, shown to help you find your place.
- `@@ -1,3 +1,4 @@` says: this block (a *hunk*) covers lines 1–3 of the old file and lines 1–4 of the new one.
- (The `index` line holds internal IDs; you can ignore it.)

Now stage the change and look again:

```
$ git add recipes.txt
$ git diff                    # prints nothing: working dir == staging area
$ git diff --staged           # staging area vs last commit
diff --git a/recipes.txt b/recipes.txt
...
+Muffins: flour, milk
```

| Command | Compares |
|---|---|
| `git diff` | Working directory *against* the staging area: changes you have *not yet staged*. |
| `git diff --staged` | Staging area against the last commit: what the next commit *would contain*. |
| `git diff HEAD` | Working directory against the last commit: *everything* changed since then. |
| `git diff A B` | Two commits, e.g. `git diff HEAD~2 HEAD`. |

**Key idea**

Run `git diff --staged` before every commit to check exactly what you are about to save. It is the best habit in this whole course.

**Try it**

With the Muffins line still staged, run `git diff HEAD`. It shows the Muffins line even though `git diff` showed nothing, because it compares the working directory with the last commit.

**Pitfall**

An empty `git diff` does not mean "nothing changed". It means nothing changed *that is not already staged*. If you just ran `git add`, look with `git diff --staged`. Also, `git diff` does not show brand-new *untracked* files; `git status` does.

# 4 Undoing mistakes

Mistakes happen at different stages, and each stage has its own fix:

| The mistake is… | Use | Safe? |
|---|---|---|
| staged by accident | `git restore --staged file` | safe |
| edited, and you want the old text back | `git restore file` | **destroys your edit** |
| in the last commit (message or a forgotten file) | `git commit --amend` | replaces that commit (old one abandoned) |
| in an older commit | `git revert commit` | safe |

We go through them one by one.

## 4.1 Unstage: `git restore --staged`

You staged a file you did not mean to. Unstaging moves it back to "modified" and *keeps your edit*:

```
$ git status --short
M  recipes.txt
$ git restore --staged recipes.txt
$ git status --short
 M recipes.txt
```

This is the command Git suggested in its status hints in Lecture 2. Nothing is lost.

### 4.2 Discard: `git restore`

**Pitfall**

**`git restore file` overwrites the file in your working directory with the staged version (or the last commit, if nothing is staged). Your uncommitted edits are gone, and there is no undo**, because Git never saved them. Look first with `git diff file`, and only restore if you are sure.

```
$ git restore recipes.txt
$ git status --short
$ cat recipes.txt
Pancakes: flour, milk, eggs
Crepes: flour, milk, eggs, butter
Waffles: flour, milk, eggs, sugar
```

The status is empty (clean) and the Muffins line is gone for good. (To *look* at an old version of a file without touching your files, use `git show HEAD~2:recipes.txt`, which prints the pancake-only version.)

### 4.3 Undo an old commit safely: `git revert`

`git revert` undoes a commit by making a *new* commit that does the opposite. Nothing is erased, so it is always safe, even for shared work. (The listing below assumes a clean `demo3` with no uncommitted changes. The new commit's ID depends on when you run it, so yours will differ from `02e79f4`, and you may also see a `Date:` line.)

```
$ git revert --no-edit HEAD
[main 02e79f4] Revert "Add waffle recipe"
 1 file changed, 1 deletion(-)
$ git log --oneline
02e79f4 Revert "Add waffle recipe"
659ed06 Add waffle recipe
c595297 Add crepe recipe
906ca29 Add pancake recipe
9d99f7a Add README
```

The waffle line is gone from `recipes.txt`, but the waffle commit is still in the history, followed by the commit that undoes it.

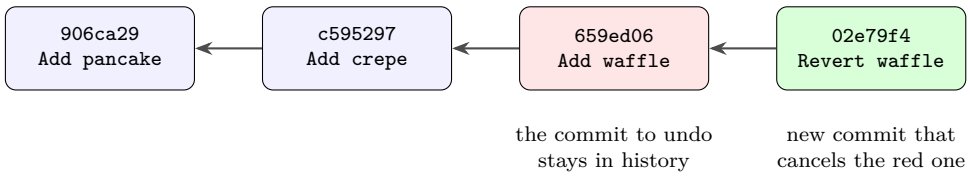



**Figure 3:** Revert adds to the history; it never removes anything. The project's *contents* after the revert equal those after `c595297`.

By default `git revert` opens an editor for the message (the same vim trap as in Lecture 2); `--no-edit` accepts the standard message `Revert "..."`. You can revert any commit by ID. One caution: if later commits changed the same lines, Git cannot undo the old commit automatically and reports a *conflict*. For example, on the original four commits of `demo3`, reverting `c595297` (the crepes) conflicts, because the waffle commit changed the neighbouring line. Run `git revert --abort` to cancel safely. Resolving conflicts is the subject of Lecture 5.

### 4.4 Fix the last commit: `git commit --amend`

You just committed and notice a typo in the message, or you forgot a file. `--amend` replaces the last commit with a corrected one. (The examples here and the revert examples above are *alternatives*: reset `demo3`, as described at the start, before trying the other one.)

```
$ git commit --amend -m "Add waffles recipe (with sugar)"
[main f1c345f] Add waffles recipe (with sugar)
 Date: Mon Mar 4 09:00:00 2024 +0000
 1 file changed, 1 insertion(+)
```

(An amended commit gets an ID and date that depend on the moment you run the command, so yours will differ from `f1c345f` and the date shown.)

To add a forgotten file to the last commit, stage it and amend while keeping the message:

```
$ echo "Syrup: sugar, water" > syrup.txt
$ git add syrup.txt
$ git commit --amend --no-edit
```

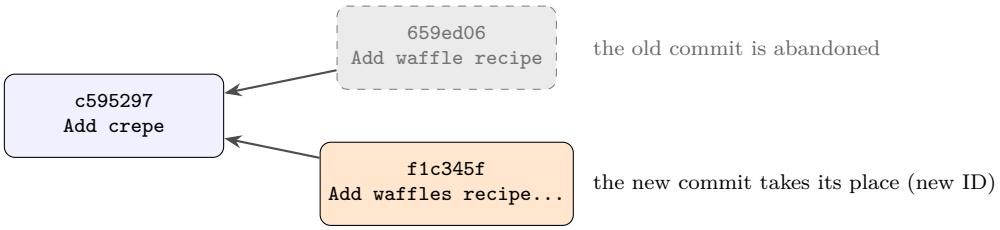



**Figure 4:** Amending does not edit a commit, it *replaces* it with a new one, which gets a new ID.

**Pitfall**

Amend only commits that nobody else has. Once you share a commit (Lecture 6 explains sharing), replacing it causes trouble for your collaborators. Rule: **amend only the most recent commit, and only if you have not shared it.** Otherwise use `revert`.

### 4.5 Deleting and untracking files

Two more commands complete the toolbox. Try them on a throw-away file:

```
$ echo "x" > scratch.txt
$ git add scratch.txt
$ git commit -m "Add scratch"
$ git rm --cached scratch.txt     # stop tracking it, keep the file on disk
rm 'scratch.txt'
$ git status --short
D  scratch.txt
?? scratch.txt
```

The first line of status (`D  `, deletion staged) says the *next commit* will no longer contain the file; the second says the file is still in your folder, now untracked. Plain `git rm scratch.txt` (without `--cached`) removes the file from your folder *and* stages the removal; commit as usual. (Deleting by hand with your file manager works too; then `git add file` stages the deletion.) `git rm --cached file` is the fix promised in Lecture 2 for a file you committed by mistake and now want in `.gitignore`: Git forgets it but leaves it on disk. It stays in the old commits, so it is no cure for a leaked secret. (`git rm` refuses to delete a file that has uncommitted changes, unless you add `-f`.)

# 5 Which tool when?

**Key idea**

Ask two questions. *Is the mistake committed yet?* If not, use `restore` (and read the warning). *If it is committed, is the commit shared?* If not, `--amend` is fine for the last one. If it may be shared, use `revert`.

Even an "abandoned" amended commit can usually be rescued, as Lecture 10 shows; that lecture also introduces `git reset`, a powerful tool we deliberately skip for now.

# 6 Homework 3

**Homework 3: Rescue the repository**

From the course folder run:

```
$ bash codeHints/L03/setup_hw3.sh
$ cd practice/hw3
```

You get a repository with four commits and a messy working directory. Run `git log --oneline`, `git status` and `git diff`/`git diff --staged` to see the situation: `list.txt` has a junk line that is not staged, and `notes.txt` has a change that is staged too early. Do the tasks in this order. If you get stuck or break something, rebuild with `bash codeHints/L03/setup_hw3.sh --reset` (from the course folder).

### 3.1 Investigate

Using only `git log` (and `git show` if you like), find the ID of the commit whose message is `Buy fireworks` and the number of commits. In the file `practice/hw3_answers.txt` (outside the repository) write two lines of the form (create the file with `echo` or `printf`, in a terminal)

```
fireworks=<7-character ID>
commits=<number>
```

### 3.2 Discard the junk

Throw away the unwanted line in `list.txt` so the file matches the last commit again.

### 3.3 Unstage

Take the change in `notes.txt` out of the staging area, but *keep the edit* in the file. Afterwards `git status --short` should show a space, then `M`, then `notes.txt` (the first column is blank).

### 3.4 Fix the typo

The latest commit message says `Update notse`. Change it to `Update notes`. The `TODO: add syrup` line must *not* end up in that commit.

### 3.5 Revert

The fireworks were a bad idea. Undo the commit `Buy fireworks` with `git revert --no-edit`, using its ID from 3.1. History must keep growing, not shrink.

### 3.6 Question

Add a third line to `practice/hw3_answers.txt`: `no-undo=<command>`, naming the one Git subcommand from this lecture that can destroy uncommitted work with no way back (just the word, for example `no-undo=log`).

The final state: five commits, a clean `list.txt`, and only `notes.txt` modified (not staged), still containing the TODO line.

**Check yourself:** from the course folder run `bash codeHints/L03/check_hw3.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L03/`).

# 7 Summary

- `git log [--oneline]` lists history; `git show c` shows one commit; `HEAD`, `HEAD~1...` name commits relative to where you are.
- `git diff` (unstaged), `git diff --staged` (what you will commit), `git diff HEAD` (everything), `git diff A B` (two commits).
- Unstage with `restore --staged`; discard edits with `restore` (permanent!); fix the last unshared commit with `commit --amend`; undo an older commit safely with `revert`.
- `git rm` deletes and stages; `git rm --cached` untracks but keeps the file.

**Further reading:** *Pro Git* (https://git-scm.com/book), Chapters 2.3 "Viewing the Commit History" and 2.4 "Undoing Things"; `git help log`, `diff`, `restore`, `revert`.

**Solutions**

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

## Solutions to Homework 3

```
$ bash codeHints/L03/setup_hw3.sh
$ cd practice/hw3
$ git log --oneline
e672c50 Update notse
6e1cc4f Buy fireworks
809893e Add recipe notes
2217470 Add shopping list
$ git restore list.txt
$ git restore --staged notes.txt
$ git commit --amend -m "Update notes"
$ git revert --no-edit 6e1cc4f
$ git status --short
 M notes.txt
$ printf 'fireworks=6e1cc4f\ncommits=4\nno-undo=restore\n' > ../hw3_answers.txt
```

(Your IDs for the first three commits match these because the setup script fixes author and dates. The amended and revert commits get new IDs.)

**3.1.** `git log --oneline` shows the ID `6e1cc4f` next to `Buy fireworks` and four lines, so `commits=4`. The answer file lives in `practice/`, outside the repository, so it does not show up as an untracked file.

**3.2.** `git diff list.txt` first shows the junk line `+oops delete me`. `git restore list.txt` removes it. This is the destructive command: it is correct here, because the line really was junk. You must do this *before* the revert, because the revert changes `list.txt` too and Git will not overwrite your uncommitted edit.

**3.3.** `git restore --staged notes.txt` moves the `TODO: add syrup` line from "staged" back to "modified". The line stays in the file. Check with `git status --short`: the first column is now empty.

**3.4.** `git commit --amend -m "Update notes"` rewrites the last commit with the corrected message. The order matters: if you amend while `notes.txt` is still staged, the TODO line is pulled into the commit. That is why 3.3 comes first. If you made that mistake, run `setup_hw3.sh --reset` and start again.

**3.5.** `git revert --no-edit 6e1cc4f` creates `Revert "Buy fireworks"` on top of the amended commit. The history has five commits now. It works without conflict because the commits after `Buy fireworks` only touched `notes.txt`.

**3.6.** The answer is `restore`. It discards uncommitted work for good. (`restore --staged` keeps your edit; `amend` leaves the old commit abandoned but recoverable, as Lecture 10 shows; and `revert` just adds a commit. None of the three deletes work from your files.)

**Common mistakes.**

- Doing 3.4 before 3.3: the TODO line gets committed. Reset and redo in the right order.
- Running `git revert` without `--no-edit` and getting stuck in the editor: press `Esc`, type `:wq`, Enter to accept the default message.
- Reverting the wrong commit (the typo commit instead of `Buy fireworks`). Look at the ID in `git log --oneline` before you revert.
- Using `git reset`, which this lecture did not teach: it can rewrite history in ways that this lecture does not cover, and the checker may flag the result.